

ADMIN PLAYBOOK

Strong Authentication — Admin Playbook for Snowflake Admins

CONTENTS

What is happening and why does it matter?

Protect Yourself First: Break-Glass Access

Step 1: Assess Your Exposure

Step 2: Fix — Remove Passwords and Convert Service Accounts

Step 3: Notify and Enroll Human Users

Step 4: Enforce with an Authentication Policy

Step 5: Disable Users Who Haven't Complied

Emergency: What to Do If a User Gets Locked Out

Ongoing Monitoring

Reference: Authentication Factor Values

See Also

Audience: Snowflake ACCOUNTADMIN or SECURITYADMIN

Purpose: Step-by-step guide to assess, fix, enforce, and monitor authentication in your Snowflake account

Updated: August 2026

This guide is intended to help you evaluate your account against Snowflake's recommended security best practices. It is not a comprehensive security audit. You are responsible for securing your own Snowflake account and determining your compliance requirements.

What is happening and why does it matter?

Snowflake is ending support for logins that use **only a username and password** — no second factor, no key, no SSO.

What that means in plain terms:

Any user or service account that connects to Snowflake with just a username and password will, after the deprecation date, be **unable to log in at all**. Queries will fail. Pipelines will stop. Dashboards will go dark.

Who is NOT affected:

- Users who already log in via SSO (Okta, Azure AD, etc.)
- Users who have already enrolled in MFA
- Service accounts using key pair (RSA JWT), OAuth, or workload identity federation

Who IS affected:

- Human users who log in with a username + password and have never set up MFA
- Service accounts (Airflow, DBT, Tableau, Python scripts, Java apps) using a username + password in their connection strings

The problem is usually smaller than it looks. Most accounts have a handful of users at risk, not hundreds.

The four-step approach:

Step 1 – Assess	Find out who is at risk (5 minutes)
Step 2 – Fix	Remove passwords from SSO users, convert service accounts
Step 3 – Notify	Email affected users, trigger MFA enrollment
Step 4 – Enforce	Apply an authentication policy so this can't regress

Before you start: Set up break-glass access for yourself first. See [Protect Yourself First](#) below. Do not skip this.

Protect Yourself First: Break-Glass Access

Before you change anything, make sure **you** can still get in if something goes wrong. This is the single most important step in this playbook.

Option 1 — Generate one-time passcodes for your admin account

```
-- Generates 5 one-time passcodes (max 10). Store these in your password manager
-- or key vault. Each can be used once as a second factor.
ALTER USER <your_admin_user> ADD MFA METHOD OTP COUNT = 5;
```

Each OTP is invalidated after use. Regenerating OTPs invalidates all previous ones.

Option 2 — Create a non-restrictive policy for admins

A user-level authentication policy **overrides** the account-level policy. Assign a permissive policy to your break-glass admin so a strict account policy can't lock you out.

```
CREATE AUTHENTICATION POLICY admin_breakglass_policy
AUTHENTICATION_METHODS = ('PASSWORD', 'SAML', 'KEYPAIR')
CLIENT_TYPES = ('SNOWFLAKE_UI', 'SNOWFLAKE_CLI', 'SNOWSQL', 'DRIVERS')
COMMENT = 'Break-glass recovery policy for administrators';

ALTER USER <your_admin_user> SET AUTHENTICATION POLICY admin_breakglass_policy;
```

Always keep at least one backup MFA method available on your admin account. If you run out of OTPs and have no other method, you will be locked out when your session expires.

Step 1: Assess Your Exposure

Run these in Snowsight as ACCOUNTADMIN or SECURITYADMIN. **Start with Query 3.**

Data notes: [ACCOUNT_USAGE](#) views have up to a 90-minute lag and retain 365 days of history. For live data covering the last 7 days, see the [INFORMATION_SCHEMA](#) query at the end of this section.

Query 1 — All Authentication Methods Per User (Last 90 Days)

```
SELECT
  user_name,
  first_authentication_factor,
  second_authentication_factor,
  COUNT(*)                               AS login_count,
  MAX(event_timestamp)                   AS last_login,
  SUM(IFF(is_success = 'YES', 1, 0))     AS successful_logins,
  SUM(IFF(is_success = 'NO', 1, 0))      AS failed_logins
FROM SNOWFLAKE.ACCOUNT_USAGE.LOGIN_HISTORY
WHERE event_timestamp >= DATEADD('day', -90, CURRENT_TIMESTAMP)
GROUP BY user_name, first_authentication_factor, second_authentication_factor
ORDER BY user_name, login_count DESC;
```

Query 2 — Users with Password-Only Logins (Last 90 Days)

```
SELECT
  user_name,
  COUNT(*)                AS password_only_logins,
  MAX(event_timestamp)    AS most_recent_password_login,
  MIN(event_timestamp)    AS first_password_login_in_window
FROM SNOWFLAKE.ACCOUNT_USAGE.LOGIN_HISTORY
WHERE event_timestamp >= DATEADD('day', -90, CURRENT_TIMESTAMP)
  AND is_success = 'YES'
  AND first_authentication_factor = 'PASSWORD'
  AND second_authentication_factor IS NULL
GROUP BY user_name
ORDER BY password_only_logins DESC;
```

Query 3 — Full User Status Report (*Start here*)

One row per user with a plain-English `status` column you can filter on.

```

WITH recent_pw_logins AS (
    SELECT
        user_name,
        COUNT(*) AS password_only_logins,
        MAX(event_timestamp) AS last_pw_only_login
    FROM SNOWFLAKE.ACCOUNT_USAGE.LOGIN_HISTORY
    WHERE event_timestamp >= DATEADD('day', -90, CURRENT_TIMESTAMP)
        AND is_success = 'YES'
        AND first_authentication_factor = 'PASSWORD'
        AND second_authentication_factor IS NULL
    GROUP BY user_name
),
recent_mfa_logins AS (
    SELECT DISTINCT user_name
    FROM SNOWFLAKE.ACCOUNT_USAGE.LOGIN_HISTORY
    WHERE event_timestamp >= DATEADD('day', -90, CURRENT_TIMESTAMP)
        AND is_success = 'YES'
        AND second_authentication_factor IS NOT NULL
)
SELECT
    u.name AS user_name,
    u.login_name,
    u.email,
    u.type AS user_type,
    u.disabled,
    u.has_password,
    u.has_mfa, -- TRUE = enrolled in any MFA
    u.has_rsa_public_key, -- TRUE = key pair auth
    u.has_pat, -- TRUE = has a programmatic
    u.has_workload_identity, -- TRUE = workload identity
    u.has_pat_access_token,
    u.has_workload_identity_federation,
    u.ext_authn_uid IS NOT NULL AS has_federated_auth,
    u.last_success_login,
    COALESCE(pw.password_only_logins, 0) AS password_only_logins_l90d,
    TO_CHAR(pw.last_pw_only_login, 'YYYY-MM-DD') AS last_pw_only_login,
    CASE
        WHEN u.disabled = TRUE THEN 'Disabled'
        WHEN u.type IN ('SERVICE', 'SERVICE_AGENT') THEN 'Service user – password
            not allowed, safe'
        WHEN u.has_password = FALSE THEN 'No password – safe'
        WHEN u.has_mfa = TRUE THEN 'MFA enrolled – safe'
        WHEN u.has_rsa_public_key = TRUE
            AND COALESCE(pw.password_only_logins, 0) = 0 THEN 'Key pair in use – remove
            password'
        WHEN pw.password_only_logins > 0 THEN 'Password-only – NEEDS
            ACTION'
        WHEN u.last_success_login < DATEADD('day', -90, CURRENT_TIMESTAMP)
            OR u.last_success_login IS NULL THEN 'Inactive – review for
            disable'
        ELSE 'Has password, no recent login – review'
    END AS status
FROM SNOWFLAKE.ACCOUNT_USAGE.USERS u
LEFT JOIN recent_pw_logins pw ON pw.user_name = u.name

```

```
LEFT JOIN recent_mfa_logins mfa ON mfa.user_name = u.name
WHERE u.deleted_on IS NULL
ORDER BY password_only_logins_l90d DESC, u.name;
```

What to do with the results:

Status	What it means	Your action
Password-only – NEEDS ACTION	Actively logging in with password, no MFA	Step 2 or 3 depending on user type
MFA enrolled – safe	HAS_MFA = TRUE	Nothing
Service user – password not allowed, safe	TYPE = SERVICE; passwords are blocked for this type	Nothing
No password – safe	No password set	Nothing
Key pair in use – remove password	Using key pair but password still set	Run ALTER USER ... UNSET PASSWORD
Inactive – review for disable	No login in 90+ days, password set	Disable if no longer needed

Two things to know when reading these results:

1. **Users already converted to TYPE = SERVICE may still show historical password logins.** Those logins happened before the conversion. Snowflake now blocks passwords for that user, so the account is compliant going forward.
2. **A user can have a key pair configured AND still be logging in with a password.** This means the key was set up but the application was never switched over. These are your easiest wins — see the query below.

Bonus query — easiest wins (key pair already configured, still using password)

These accounts already have an RSA key registered in Snowflake. Someone just needs to update the application's connection string. No key generation required.

```

SELECT
    u.name,
    u.type,
    u.has_rsa_public_key,
    COUNT(h.event_timestamp)      AS password_logins_l90d,
    MAX(h.event_timestamp)        AS last_password_login
FROM SNOWFLAKE.ACCOUNT_USAGE.USERS u
JOIN SNOWFLAKE.ACCOUNT_USAGE.LOGIN_HISTORY h
    ON h.user_name = u.name
   AND h.event_timestamp >= DATEADD('day', -90, CURRENT_TIMESTAMP)
   AND h.is_success = 'YES'
   AND h.first_authentication_factor = 'PASSWORD'
   AND h.second_authentication_factor IS NULL
WHERE u.deleted_on IS NULL
   AND u.disabled = FALSE
   AND u.has_rsa_public_key = TRUE      -- key pair already registered
   AND u.has_password = TRUE           -- but password still set and in use
GROUP BY u.name, u.type, u.has_rsa_public_key
ORDER BY password_logins_l90d DESC;

```

Query 4 — Executive Summary by Authentication Method

```

SELECT
    CASE
        WHEN second_authentication_factor IS NOT NULL
            (password + second factor)'      THEN 'MFA'
        WHEN first_authentication_factor = 'PASSWORD'
            AND second_authentication_factor IS NULL
            only (NO MFA)'                  THEN 'Password-
        WHEN first_authentication_factor = 'RSA_KEYPAIR'
            auth'                          THEN 'Key pair
        WHEN first_authentication_factor = 'SAML2_ASSERTION'
            SAML'                          THEN 'SSO /
        WHEN first_authentication_factor = 'OAUTH_ACCESS_TOKEN'
            THEN 'OAuth'
        WHEN first_authentication_factor = 'PROGRAMMATIC_ACCESS_TOKEN'
            THEN 'PAT'
        WHEN first_authentication_factor = 'WORKLOAD_IDENTITY'
            identity federation'           THEN 'Workload
        WHEN first_authentication_factor = 'ID_TOKEN'
            ID token'                     THEN 'Cached SSO
        WHEN first_authentication_factor = 'WEB_SESSION'
            web session'                  THEN 'Existing
        ELSE COALESCE(first_authentication_factor, 'UNKNOWN')
    END
    COUNT(DISTINCT user_name)      AS auth_method,
    COUNT(*)                       AS distinct_users,
    COUNT(*)                       AS total_logins
FROM SNOWFLAKE.ACCOUNT_USAGE.LOGIN_HISTORY
WHERE event_timestamp >= DATEADD('day', -90, CURRENT_TIMESTAMP)
   AND is_success = 'YES'
GROUP BY 1
ORDER BY total_logins DESC;

```

Near Real-Time: Last 6 Days (No Data Lag)

```
-- Requires a database context to be set (any database – INFORMATION_SCHEMA
-- is available in every database). If you get "Invalid identifier", run
-- USE DATABASE <any_db>; first.
```

```
SELECT
    user_name,
    first_authentication_factor,
    second_authentication_factor,
    event_timestamp,
    client_ip,
    reported_client_type
FROM TABLE(INFORMATION_SCHEMA.LOGIN_HISTORY(
    DATEADD('day', -6, CURRENT_TIMESTAMP),
    CURRENT_TIMESTAMP
))
WHERE first_authentication_factor = 'PASSWORD'
    AND second_authentication_factor IS NULL
    AND is_success = 'YES'
ORDER BY event_timestamp DESC;
```

Note: The table function's retention window is exactly 7 days, but `DATEADD('day', -7, ...)` can trip a `"Cannot retrieve data from more than 7 days ago"` error depending on when the query executes relative to midnight boundaries. Use `-6 days` to stay safely inside the window.

Step 2: Fix — Remove Passwords and Convert Service Accounts

2a. Remove passwords from SSO/federated users

This is often the quickest win. Many users who log in via your identity provider also have a leftover Snowflake password. Removing it makes them compliant with no action on their part.

Find them:

```
SELECT
    name,
    login_name,
    email,
    has_mfa,
    has_rsa_public_key,
    last_success_login
FROM SNOWFLAKE.ACCOUNT_USAGE.USERS
WHERE deleted_on IS NULL
    AND disabled = FALSE
    AND has_password = TRUE
    AND ext_authn_uid IS NOT NULL -- federated auth configured
ORDER BY last_success_login DESC NULLS LAST;
```

Cross-check against actual SSO usage — confirm they really are logging in via SSO before removing the password:

```
SELECT DISTINCT user_name, first_authentication_factor, MAX(event_timestamp) AS
    last_sso_login
FROM SNOWFLAKE.ACCOUNT_USAGE.LOGIN_HISTORY
WHERE event_timestamp >= DATEADD('day', -90, CURRENT_TIMESTAMP)
    AND is_success = 'YES'
    AND first_authentication_factor IN ('SAML2_ASSERTION', 'ID_TOKEN',
    'OAUTH_ACCESS_TOKEN')
GROUP BY user_name, first_authentication_factor
ORDER BY user_name;
```

Remove the password — single user:

```
ALTER USER <username> UNSET PASSWORD;
```

Remove in bulk — only for users confirmed on the list above:

```
EXECUTE IMMEDIATE $$
DECLARE
    sso_users CURSOR FOR
        SELECT name
        FROM SNOWFLAKE.ACCOUNT_USAGE.USERS
        WHERE deleted_on IS NULL
            AND disabled = FALSE
            AND has_password = TRUE
            AND ext_authn_uid IS NOT NULL;
    uname VARCHAR;
    removed INT DEFAULT 0;
BEGIN
    FOR rec IN sso_users DO
        uname := rec.name;
        EXECUTE IMMEDIATE 'ALTER USER "' || uname || '" UNSET PASSWORD';
        removed := removed + 1;
    END FOR;
    RETURN 'Passwords removed from ' || removed || ' SSO users';
END;
$$;
```

Safety: Unsetting a password for a user with no SSO and no key pair locks them out immediately. Verify the list first.

2b. Convert service accounts to `TYPE = SERVICE`

This is the cleanest fix for service accounts. **Users with `TYPE = SERVICE` cannot have a password at all** — Snowflake blocks the property. Once converted, the account is permanently compliant.

`LEGACY_SERVICE` is itself being deprecated, so this is the recommended destination.

Find service accounts still on password auth:

```
SELECT
  u.name,
  u.type,
  u.has_password,
  u.has_rsa_public_key,
  u.last_success_login,
  COUNT(h.event_timestamp) AS pw_logins_l90d
FROM SNOWFLAKE.ACCOUNT_USAGE.USERS u
LEFT JOIN SNOWFLAKE.ACCOUNT_USAGE.LOGIN_HISTORY h
  ON h.user_name = u.name
  AND h.event_timestamp >= DATEADD('day', -90, CURRENT_TIMESTAMP)
  AND h.is_success = 'YES'
  AND h.first_authentication_factor = 'PASSWORD'
WHERE u.deleted_on IS NULL
  AND u.disabled = FALSE
  AND u.type IN ('LEGACY_SERVICE', 'SERVICE')
  AND u.has_password = TRUE
GROUP BY u.name, u.type, u.has_password, u.has_rsa_public_key, u.last_success_login
ORDER BY pw_logins_l90d DESC;
```

Migration sequence for each service account — do these in order:

```
-- 1. Generate an RSA key pair (run in your shell, not Snowflake):
-- openssl genrsa 2048 | openssl pkcs8 -topk8 -inform PEM -out rsa_key.p8 -nocrypt
-- openssl rsa -in rsa_key.p8 -pubout -out rsa_key.pub

-- 2. Register the public key (paste contents of rsa_key.pub without header/footer lines)
ALTER USER <svc_user> SET RSA_PUBLIC_KEY = '<public_key_body>';

-- 3. Update the application connection to use authenticator=snowflake_jwt + the private
    key.
-- TEST IT. Confirm the app connects successfully before continuing.

-- 4. Verify key pair logins are appearing:
SELECT event_timestamp, first_authentication_factor, is_success
FROM SNOWFLAKE.ACCOUNT_USAGE.LOGIN_HISTORY
WHERE user_name = '<svc_user>'
  AND event_timestamp >= DATEADD('hour', -24, CURRENT_TIMESTAMP)
ORDER BY event_timestamp DESC;

-- 5. Once confirmed, convert the type. This permanently blocks password auth:
ALTER USER <svc_user> SET TYPE = SERVICE;
```

Converting to `TYPE = SERVICE` also blocks `RESET PASSWORD` and `DISABLE_MFA` for that user. If you convert back to `PERSON`, the previously stored password properties are restored.

Key pair authentication is supported by all major connectors: Python, JDBC, ODBC, Go, .NET, Node.js, and Snowpark.

- Key pair setup: <https://docs.snowflake.com/en/user-guide/key-pair-auth>
- Python Connector: <https://docs.snowflake.com/en/developer-guide/python-connector/python-connector-connect>
- JDBC: <https://docs.snowflake.com/en/developer-guide/jdbc/jdbc-configure>

Step 3: Notify and Enroll Human Users

3a. Trigger Snowflake's built-in enrollment prompt

Snowflake can email the user an enrollment link directly. This is more reliable than asking them to navigate menus.

```
-- Sends the user an email with an MFA enrollment link.  
-- If they have no verified email, this returns a URL you can send them manually.  
ALTER USER <username> ENROLL MFA;
```

Bulk enrollment prompt for all password-only human users:

```
EXECUTE IMMEDIATE $$  
DECLARE  
    pw_users CURSOR FOR  
        SELECT DISTINCT u.name  
        FROM SNOWFLAKE.ACCOUNT_USAGE.USERS u  
        INNER JOIN (  
            SELECT DISTINCT user_name  
            FROM SNOWFLAKE.ACCOUNT_USAGE.LOGIN_HISTORY  
            WHERE event_timestamp >= DATEADD('day', -90, CURRENT_TIMESTAMP)  
            AND is_success = 'YES'  
            AND first_authentication_factor = 'PASSWORD'  
            AND second_authentication_factor IS NULL  
        ) pw ON pw.user_name = u.name  
        WHERE u.deleted_on IS NULL  
        AND u.disabled = FALSE  
        AND u.has_mfa = FALSE  
        AND (u.type = 'PERSON' OR u.type IS NULL);  
    uname VARCHAR;  
    prompted INT DEFAULT 0;  
BEGIN  
    FOR rec IN pw_users DO  
        uname := rec.name;  
        EXECUTE IMMEDIATE 'ALTER USER "' || uname || '" ENROLL MFA';  
        prompted := prompted + 1;  
    END FOR;  
    RETURN 'Enrollment prompts sent: ' || prompted;  
END;  
$$;
```

Verify what a user has enrolled:

```
SHOW MFA METHODS FOR USER <username>;
```

3b. Email template

Copy and customize before sending.

Subject: Action required — Update your Snowflake login before [DATE]

Hi [First Name],

We're making a required security update to how you log in to Snowflake. Starting **[DATE]**, Snowflake will no longer accept logins using only a username and password. You'll need to set up a second verification step, called MFA.

If you already use single sign-on ([Company SSO, e.g. Okta]) to reach Snowflake, you're all set — no action needed.

If you log in with a Snowflake username and password, here's what to do before [DATE]:

1. Sign in to [your Snowsight URL]
2. Click your name in the bottom-left corner, then **Settings**
3. Click **Authentication**, then enroll an MFA method
4. Choose one:
 - **Passkey** (recommended — your phone, fingerprint, or a hardware key)
 - **Authenticator app** — Google Authenticator, Microsoft Authenticator, or Authy
 - **Duo** — if [Company] uses Duo

Takes about 2 minutes. After enrolling, you'll be prompted for the second factor when you sign in.

Questions? Reply to this email or contact [admin email].

Thank you, [Your name] [Company] Snowflake Administrator

3c. Stored procedure to bulk-email affected users

Requires an email notification integration. Verify you have one:

```
SHOW INTEGRATIONS;  
-- Look for TYPE = NOTIFICATION, ENABLED = true  
-- Setup docs: https://docs.snowflake.com/en/user-guide/notifications/email-stored-proc
```

```

CREATE OR REPLACE PROCEDURE notify_password_only_users(
    notification_integration VARCHAR,
    deadline_date            VARCHAR,
    admin_email              VARCHAR
)
RETURNS VARCHAR
LANGUAGE SQL
AS
$$
DECLARE
    at_risk_users CURSOR FOR
        SELECT DISTINCT u.name AS uname, u.email AS uemail
        FROM SNOWFLAKE.ACCOUNT_USAGE.USERS u
        INNER JOIN (
            SELECT DISTINCT user_name
            FROM SNOWFLAKE.ACCOUNT_USAGE.LOGIN_HISTORY
            WHERE event_timestamp >= DATEADD('day', -90, CURRENT_TIMESTAMP)
            AND is_success = 'YES'
            AND first_authentication_factor = 'PASSWORD'
            AND second_authentication_factor IS NULL
        ) pw ON pw.user_name = u.name
        WHERE u.deleted_on IS NULL
        AND u.disabled = FALSE
        AND u.has_mfa = FALSE
        AND (u.type = 'PERSON' OR u.type IS NULL) -- human users only
        AND u.email IS NOT NULL;
    sent INT DEFAULT 0;
BEGIN
    FOR rec IN at_risk_users DO
        CALL SYSTEM$SEND_EMAIL(
            :notification_integration,
            rec.uemail,
            'Action Required: Update Your Snowflake Login Before ' || :deadline_date,
            'Hi ' || rec.uname || ', ' || CHR(10) || CHR(10) ||
            'Your Snowflake account uses password-only authentication. Starting ' ||
            :deadline_date || ', Snowflake will require a second verification step
            (MFA).' ||
            CHR(10) || CHR(10) ||
            'To enroll: sign in to Snowsight, click your name in the bottom-left, ' ||
            'then Settings > Authentication > enroll an MFA method.' || CHR(10) ||
            CHR(10) ||
            'Options: Passkey (recommended), Authenticator App, or Duo.' || CHR(10) ||
            CHR(10) ||
            'If you already use SSO to reach Snowflake, no action is needed.' ||
            CHR(10) || CHR(10) ||
            'Questions? Contact ' || :admin_email || '.' || CHR(10) || CHR(10) ||
            'Your Snowflake Administrator'
        );
        sent := sent + 1;
    END FOR;
    RETURN 'Emails sent: ' || sent;
END;
$$;

-- Run it:

```

```
CALL notify_password_only_users(
  'my_email_notification_integration',
  'October 15, 2026',
  'snowflake-admin@yourcompany.com'
);
```

Users need email addresses populated: `ALTER USER <name> SET EMAIL = '<email>';`

Step 4: Enforce with an Authentication Policy

Monitoring and emails alone won't get you to compliance — users can ignore them indefinitely. An **authentication policy** makes MFA mandatory at the platform level.

Confirm your break-glass access works before applying an account-level policy. See [Protect Yourself First](#).

Require MFA for password users (SSO users exempt)

```
CREATE AUTHENTICATION POLICY require_mfa_policy
  MFA_ENROLLMENT = 'REQUIRED'
  CLIENT_TYPES = ('SNOWFLAKE_UI', 'DRIVERS', 'SNOWFLAKE_CLI', 'SNOWSQL')
  MFA_POLICY = ( ENFORCE_MFA_ON_EXTERNAL_AUTHENTICATION = 'NONE' )
  COMMENT = 'Require MFA enrollment for password users';
```

Require MFA for password AND SSO users

```
CREATE AUTHENTICATION POLICY require_mfa_everywhere_policy
  MFA_ENROLLMENT = 'REQUIRED'
  CLIENT_TYPES = ('SNOWFLAKE_UI', 'DRIVERS', 'SNOWFLAKE_CLI', 'SNOWSQL')
  MFA_POLICY = ( ENFORCE_MFA_ON_EXTERNAL_AUTHENTICATION = 'ALL' )
  COMMENT = 'Require MFA for all human users including SSO';
```

Restrict which MFA methods are allowed

```
CREATE AUTHENTICATION POLICY passkey_and_totp_only
  MFA_ENROLLMENT = 'REQUIRED'
  CLIENT_TYPES = ('SNOWFLAKE_UI', 'DRIVERS')
  MFA_POLICY = ( ALLOWED_METHODS = ('PASSKEY', 'TOTP') )
  COMMENT = 'Passkey or authenticator app only, no Duo';
```

Apply the policy

```
-- Account-wide:
ALTER ACCOUNT SET AUTHENTICATION POLICY require_mfa_policy;

-- Or to a single user (user-level overrides account-level):
ALTER USER <username> SET AUTHENTICATION POLICY require_mfa_policy;

-- Or to all service users only:
ALTER ACCOUNT SET AUTHENTICATION POLICY <policy_name> FOR ALL SERVICE USERS;
```

Block password auth entirely for service accounts

```
CREATE AUTHENTICATION POLICY service_no_password_policy
  AUTHENTICATION_METHODS = ('KEYPAIR', 'OAUTH', 'WORKLOAD_IDENTITY',
    'PROGRAMMATIC_ACCESS_TOKEN')
  COMMENT = 'Service accounts cannot use passwords';

ALTER ACCOUNT SET AUTHENTICATION POLICY service_no_password_policy FOR ALL SERVICE
  USERS;
```

Verify where a policy is applied

```
-- Users assigned to a specific policy
SELECT * FROM TABLE(
  <your_db>.INFORMATION_SCHEMA.POLICY_REFERENCES(
    POLICY_NAME => '<your_db>.<your_schema>.require_mfa_policy'
  )
);

-- Which policy applies to a given user
SELECT * FROM TABLE(
  <your_db>.INFORMATION_SCHEMA.POLICY_REFERENCES(
    REF_ENTITY_DOMAIN => 'USER', REF_ENTITY_NAME => '<username>'
  )
);
```

Important constraints:

- `MFA_ENROLLMENT = 'REQUIRED'` requires `SNOWFLAKE_UI` in `CLIENT_TYPES` — Snowsight is the only place users can enroll.
- Omitting `DRIVERS` from `CLIENT_TYPES` will break automated ingestion.
- `MFA_ENROLLMENT` accepts `'REQUIRED'`, `'REQUIRED_PASSWORD_ONLY'`, or `'OPTIONAL'`.
`OPTIONAL` exists only for backward compatibility and will not remain honored.

Step 5: Disable Users Who Haven't Complied

Find users still on password-only after your notification date

```
SELECT DISTINCT
  h.user_name,
  u.email,
  u.type AS user_type,
  u.has_mfa,
  COUNT(*) AS pw_only_logins_since_notification,
  MAX(h.event_timestamp) AS last_pw_only_login
FROM SNOWFLAKE.ACCOUNT_USAGE.LOGIN_HISTORY h
JOIN SNOWFLAKE.ACCOUNT_USAGE.USERS u ON u.name = h.user_name
WHERE h.event_timestamp >= '2026-09-01':TIMESTAMP -- your notification date
  AND h.is_success = 'YES'
  AND h.first_authentication_factor = 'PASSWORD'
  AND h.second_authentication_factor IS NULL
  AND u.deleted_on IS NULL
  AND u.disabled = FALSE
GROUP BY h.user_name, u.email, u.type, u.has_mfa
ORDER BY last_pw_only_login DESC;
```

Disable / re-enable a single user

```
ALTER USER <username> SET DISABLED = TRUE;

-- Re-enable:
ALTER USER <username> SET DISABLED = FALSE;
```

Bulk disable — dry run first

The `ALTER` is commented out. Run as-is to see the count, then uncomment to execute.

```

EXECUTE IMMEDIATE $$
DECLARE
    non_compliant CURSOR FOR
        SELECT DISTINCT h.user_name AS uname
        FROM SNOWFLAKE.ACCOUNT_USAGE.LOGIN_HISTORY h
        JOIN SNOWFLAKE.ACCOUNT_USAGE.USERS u ON u.name = h.user_name
        WHERE h.event_timestamp >= '2026-09-01':TIMESTAMP
            AND h.is_success = 'YES'
            AND h.first_authentication_factor = 'PASSWORD'
            AND h.second_authentication_factor IS NULL
            AND u.deleted_on IS NULL
            AND u.disabled = FALSE
            AND u.has_mfa = FALSE
            AND (u.type = 'PERSON' OR u.type IS NULL);
    affected INT DEFAULT 0;
    names VARCHAR DEFAULT '';
BEGIN
    FOR rec IN non_compliant DO
        -- UNCOMMENT THE NEXT LINE TO ACTUALLY DISABLE:
        -- EXECUTE IMMEDIATE 'ALTER USER "' || rec.uname || '" SET DISABLED = TRUE';
        affected := affected + 1;
        names := names || rec.uname || ', ';
    END FOR;
    RETURN 'Count: ' || affected || ' | Users: ' || names;
END;
$$;

```

Never bulk-disable service accounts. Migrate them to key pair auth first (Step 2b). Disabling a service account breaks production pipelines immediately.

Emergency: What to Do If a User Gets Locked Out

Human user can't log in

1. Check why:

```

SELECT event_timestamp, user_name, is_success, error_code, error_message,
       first_authentication_factor, second_authentication_factor, reported_client_type
FROM SNOWFLAKE.ACCOUNT_USAGE.LOGIN_HISTORY
WHERE user_name = '<username>'
ORDER BY event_timestamp DESC
LIMIT 10;

```

2. Temporarily bypass MFA — the supported way to get them working immediately:

```

-- Allows single-factor password login for 30 minutes
ALTER USER <username> SET MINS_TO_BYPASS_MFA = 30;

```

3. Have them enroll during that window:

```
ALTER USER <username> ENROLL MFA;
```

4. If they lost their MFA device — clear their methods and force re-enrollment:

```
-- Clears existing MFA methods; user is prompted to add a new one on next sign-in  
ALTER USER <username> SET DISABLE_MFA = TRUE;  
ALTER USER <username> ENROLL MFA;
```

5. If the account was disabled:

```
ALTER USER <username> SET DISABLED = FALSE;
```

Service account connection is failing

1. Check the error:

```
SELECT event_timestamp, user_name, error_code, error_message,  
       first_authentication_factor, reported_client_type, reported_client_version  
FROM SNOWFLAKE.ACCOUNT_USAGE.LOGIN_HISTORY  
WHERE user_name = '<svc_user>'  
ORDER BY event_timestamp DESC  
LIMIT 10;
```

2. Fastest fix — set up key pair auth:

```
openssl genrsa 2048 | openssl pkcs8 -topk8 -inform PEM -out rsa_key.p8 -nocrypt  
openssl rsa -in rsa_key.p8 -pubout -out rsa_key.pub
```

```
ALTER USER <svc_user> SET RSA_PUBLIC_KEY = '<contents_of_rsa_key.pub>';
```

Then update the application to `authenticator=snowflake_jwt` with the private key path.

3. Interim option — a programmatic access token (short-lived, useful while wiring up key pair):

```
ALTER USER <svc_user> ADD PROGRAMMATIC ACCESS TOKEN my_temp_token  
DAYS_TO_EXPIRY = 7  
COMMENT = 'Temporary token during key pair migration';
```

Note: PAT usage requires the user to be covered by a network policy by default. See `PAT_POLICY` in the authentication policy docs to adjust.

You are locked out of your own admin account

1. Use a stored one-time passcode (OTP) if you generated them — see [Protect Yourself First](#)
 2. Ask another ACCOUNTADMIN to run `ALTER USER <you> SET MINS_TO_BYPASS_MFA = 30;`
 3. If no other admin can help, contact **Snowflake Support**: <https://community.snowflake.com/s/article/How-to-Submit-a-Support-Case>
-

Ongoing Monitoring

Wrap the check in a stored procedure and have a task call it. (A task body should call a procedure rather than embed a scripting block.)

```

-- 1. The check procedure
CREATE OR REPLACE PROCEDURE check_password_only_logins(
    notification_integration VARCHAR,
    admin_email              VARCHAR
)
RETURNS VARCHAR
LANGUAGE SQL
AS
$$
DECLARE
    pw_only_count INT;
BEGIN
    SELECT COUNT(DISTINCT user_name) INTO :pw_only_count
    FROM SNOWFLAKE.ACCOUNT_USAGE.LOGIN_HISTORY
    WHERE event_timestamp >= DATEADD('day', -7, CURRENT_TIMESTAMP)
    AND is_success = 'YES'
    AND first_authentication_factor = 'PASSWORD'
    AND second_authentication_factor IS NULL;

    IF (pw_only_count > 0) THEN
        CALL SYSTEM$SEND_EMAIL(
            :notification_integration,
            :admin_email,
            'Snowflake Auth Alert: Password-Only Logins Detected',
            pw_only_count || ' user(s) logged in with password-only authentication '
            || 'in the last 7 days. Run Query 3 from the Strong Auth playbook for
            details.'
        );
    END IF;
    RETURN 'Password-only users found: ' || pw_only_count;
END;
$$;

-- 2. The task that calls it
CREATE OR REPLACE TASK monitor_password_only_logins
    WAREHOUSE = <your_warehouse>
    SCHEDULE = 'USING CRON 0 8 * * 1 America/New_York'
AS
    CALL check_password_only_logins('<your_email_integration>',
    '<admin@yourcompany.com>');

ALTER TASK monitor_password_only_logins RESUME;

```

Reference: Authentication Factor Values

These are the values that actually appear in `LOGIN_HISTORY`.

FIRST_AUTHENTICATION_FACTOR

Value	Description
PASSWORD	Username + password
RSA_KEYPAIR	Key pair (JWT) — recommended for service accounts
SAML2_ASSERTION	SAML SSO (Okta, Entra ID, etc.)
OAuth_ACCESS_TOKEN	OAuth
PROGRAMMATIC_ACCESS_TOKEN	PAT
WORKLOAD_IDENTITY	Workload identity federation (AWS/Azure/GCP/OIDC)
ID_TOKEN	Cached SSO token (connection caching)
WEB_SESSION	Existing browser session

SECOND_AUTHENTICATION_FACTOR

Value	Description
NULL	No MFA — this is what you're looking for
PASSKEY	Passkey (WebAuthn) — hardware key, device biometric, or password manager
TOTP	Authenticator app (Google Authenticator, Microsoft Authenticator, Authy)
DUO_PUSH	Duo push notification
MFA_PROMPT	Generic MFA challenge completed (seen on some client types where the specific method isn't reported)
MFA_TOKEN	Cached MFA token (valid up to 4 hours; requires <code>ALLOW_CLIENT_MFA_CACHING</code>)

The specific method (`PASSKEY` , `TOTP` , `DUO_PUSH`) is usually reported directly rather than a generic `MFA_PROMPT` value — the exact set of values you see can vary slightly by client and Snowflake release. Any non-NULL value in this column means MFA was satisfied. To see exactly which methods a specific user has enrolled (independent of login history), use `SHOW MFA METHODS FOR USER <name>;`.

User TYPE values

Value	Description	Password allowed?
<code>PERSON</code> (or NULL)	Human user	Yes — must enroll MFA
<code>SERVICE</code>	Service/application	No — blocked by Snowflake
<code>SERVICE_AGENT</code>	Automated AI agent	No
<code>LEGACY_SERVICE</code>	Legacy non-interactive integration	Yes — being deprecated, migrate to SERVICE
<code>SNOWFLAKE_SERVICE</code>	Snowflake-internal	N/A

See Also

- [Planning for the deprecation of single-factor password sign-ins](#)
- [Authentication policies](#)
- [CREATE AUTHENTICATION POLICY](#)
- [ALTER USER](#)
- [Key_pair authentication](#)
- [LOGIN_HISTORY view](#)
- [Types of users](#)
- [Email notifications \(SYSTEM\\$SEND_EMAIL\)](#)